

matrix, we can define $n=7$ and then create a matrix with n rows and n columns:

```
n = 7
square = [[float('nan') for i in range(0,n)] for j in range(0,n)]
```

In this case, we don't know what numbers to put in the square, so we fill it entirely with entries equal to `float('nan')`. Here, `nan` stands for *not a number*, which we can use as a placeholder in Python when we want to fill up a list before we know what numbers to use. If we run `print(square)`, we find that this matrix by default is filled with `nan` entries:

```
[nan, nan, nan, nan, nan, nan, nan], [nan, nan, nan, nan, nan, nan, nan], [nan,
nan, nan, nan, nan, nan, nan], [nan, nan, nan, nan, nan, nan, nan], [nan, nan,
nan, nan, nan, nan, nan], [nan, nan, nan, nan, nan, nan, nan], [nan, nan, nan,
nan, nan, nan, nan]]
```

This square is not too pretty as it is output in the Python console, so we can write a function that will print it in a more readable way:

```
def printsquare(square):
    labels = ['['+str(x)+']' for x in range(0,len(square))]
    format_row = "{:>6}" * (len(labels) + 1)
    print(format_row.format("", *labels))
    for label, row in zip(labels, square):
        print(format_row.format(label, *row))
```

Don't worry about the details of the `printsquare()` function, since it's only for pretty printing and not part of our algorithm. We can fill in the central five squares with simple commands. First, we can get the indices of the central entry as follows:

```
import math
center_i = math.floor(n/2)
center_j = math.floor(n/2)
```

The central five squares can be populated according to the expressions in [Table 2-10](#) as follows:

```
square[center_i][center_j] = int((n**2 + 1)/2)
square[center_i + 1][center_j] = 1
square[center_i - 1][center_j] = n**2
```
